Jacob Hough

CS 320 – Project Two

Summary and Reflections Report

Summary

For Project One, I developed and tested three back-end services: Contact, Task, and Appointment. My approach was to turn the requirements for each class into JUnit tests, including tests for both valid information and information the program should reject. Contact required a unique, unchangeable ID, name and address length limits, and a ten-digit phone number. Task required its own ID, name, and description limits. Appointment required a unique ID, a description within its limit, and a date that could not be in the past. The service tests also checked adding and deleting records, along with updating the Contact and Task records. These tests were directly tied to what the assignment asked the software to do.

I tried to test more than whether a method could run without crashing. For example, in the testAddContact() method of ContactServiceTest.java, I used assertEquals(contact, service.getContact("12345")) after adding a contact. This checked that the service actually stored the object. In the delete tests, I checked that the record was gone and that deleting the same ID again threw an IllegalArgumentException. I used the same approach with Task and Appointment. In TaskServiceTest, the update test checked the name and description with assertEquals after changing them. Keeping these checks tied to specific requirements made the results easier to understand when a test failed.

The coverage results were encouraging, but they also taught me to be careful about what coverage means. Eclipse showed 100% coverage for Contact.java and ContactService.java, 88.1% for Task.java, and 100% for TaskService.java. The Appointment project showed 87.8% overall coverage. Eclipse also reported 79.2% overall for the Contact project because it included the JUnit

test files in that calculation, even though both application classes reached 100%. These figures show that the tests exercised much of the application code, but coverage alone cannot prove that every requirement is tested thoroughly.

My instructor's feedback made that limitation especially clear. He changed several maximum field lengths by one character, yet my tests still passed. I had checked values that were much too long, but I had not checked values exactly at the allowed limit or just one character beyond it. An 11-digit phone number also needed its own test. This showed me that a test can execute a line of code without catching a subtle defect in its logic. I would strengthen the suite with boundary-value tests for the exact 10-, 20-, 30-, and 50-character limits, plus values immediately beyond those limits.

Writing the tests also helped me see what technically sound and efficient tests look like. I used assertThrows(IllegalArgumentException.class, ...) to confirm that invalid inputs were rejected, and assertNull(service.getTask("12345")) to confirm a deletion. In the testDeleteTask() method of TaskServiceTest.java, assertNull(service.getTask("12345")) checks the result directly. Keeping this test focused on one task made it quick to rerun and a failure easier to trace. I did not need to build a larger collection of unrelated tasks to check that one behavior. At the same time, the feedback identified improvements: my Contact update test kept the last name as "Hough," so it did not prove that field could change; using "Smith" would be better. I also need tests for null and overlong Task setter inputs and consistent method naming, including setLastName.

Reflection

The main technique I used was unit testing with JUnit. Unit tests examine individual classes or methods and compare the results against expected behavior. I also used positive and negative testing by trying acceptable inputs and inputs that should cause exceptions. Boundary-value analysis would have been a particularly useful addition because many of the requirements were expressed as exact length limits. Instead of testing only an obviously invalid long name, I should

test a valid ten-character name and an invalid eleven-character name. That approach is useful in forms, account systems, and other programs with strict input rules.

I did not perform full integration, system, or acceptance testing. Integration testing checks how components communicate, such as a service connecting to a database; it could uncover errors in how records are saved or retrieved. System testing examines the finished application as a whole, including its user interface and external services, and can expose problems that isolated unit tests miss. Acceptance testing asks whether the completed software satisfies the customer's needs. For example, a customer could try scheduling, viewing, and canceling appointments in the finished app to confirm that the workflow meets their expectations, even if its individual methods pass JUnit tests. Project One only required in-memory back-end services, so unit testing was the practical focus, although broader testing would be necessary before a real customer release.

I learned that caution means looking for ways the software might fail, not just finding evidence that my code works. For example, I tested duplicate IDs and attempts to delete records that did not exist. However, I overlooked cases right at the boundaries of the validation rules. The classes and services also depend on each other: a service might correctly store an object while the object itself accepts invalid data. That is why both layers need tests. I would now pay more attention to unusual inputs and to how a change in one method could affect another.

Bias is another concern when testing my own work. Because I knew how I intended the code to behave, I tended to choose inputs that confirmed my expectations. The last-name update test is a good example: using the same original and updated value made the test look successful without actually proving an update happened. My instructor's modified versions of the code exposed assumptions I had missed. In future projects, I would use a requirement-by-requirement checklist, deliberately test edge cases, and ask someone else to review important tests when possible.

Finally, I think being disciplined about quality means not stopping as soon as the tests turn green. Shortcuts can leave weaknesses that become harder to fix as more features are added. In this project, adding stronger assertions after earlier milestone feedback improved the add and delete tests. Following consistent naming, adding missing setter tests, and checking exact boundaries would be the next steps. I would also keep tests updated whenever requirements change and run them again after modifying code. That would help limit technical debt by catching problems while the code is still small and understandable, rather than passing them along to a larger system.